LAB ASSIGNMENT VI

NATURAL LANGUAGE PROCESSING (PMDS606P)

CLASS ID - VL2025260105170

MAXIMUM MARKS: 10 **DUE DATE**: 07.11.2025

1. Problem Statement 1:

You are given a URL of a web page (for example:
🔗 https://www.bbc.com/news/technology-67232835)

Your task is to perform **basic Natural Language Processing (NLP)** operations on the text content of this web page.

Tasks to Perform:

1. **Extract text content** from the given URL (ignore HTML tags, advertisements, and navigation text).
2. **Tokenize** the text into words.
3. **Count the total number of tokens (words)**.
4. **Find the number of unique words** in the text.
5. **Compute the frequency of each word** (ignore case and punctuation).
6. **Identify the word(s) with the maximum frequency.**
7. **(Optional)**: Display the top 10 most frequent words in descending order.

Example Output:

For the given BBC Technology article, your output might look like:

```
Total Tokens: 1520
Unique Words: 680
Most Frequent Word: "the" (appeared 96 times)
Top 10 Words:
1. the – 96
2. to – 58
3. and – 54
4. of – 48
5. a – 45
6. in – 42
7. for – 26
8. on – 21
9. that – 18
10. is – 17
```

---

2. Problem Statement 2:

Part-of-Speech (POS) Tagging using Hidden Markov Model

Problem Statement:

You are given a set of English sentences. Each word in a sentence needs to be assigned its **Part of Speech (POS)** — such as noun, verb, adjective, adverb, etc.

Since the actual POS tags are *hidden* and only the words are *observed*, this can be modeled as a **Hidden Markov Model (HMM)** problem.

Given:

You have a training dataset consisting of pairs of:

- A **word** (observation)
- Its **POS tag** (hidden state)

Example training data:

Word	POS Tag
The	DET
cat	NOUN
sat	VERB
on	PREP
the	DET
mat	NOUN

Tasks to Perform:

1. **Define the HMM components:**
    - Set of *hidden states* = POS tags (e.g., {NOUN, VERB, ADJ, DET, PREP})
    - Set of *observations* = words in sentences
    - *Transition probabilities* between states (e.g., `P(VERB | NOUN)`)
    - *Emission probabilities* (e.g., `P(cat | NOUN)`)
    - *Initial probabilities* of states (e.g., `P(DET)`)
2. **Estimate probabilities** from the training data.
3. Given a **new sentence without tags**, e.g.:
4. `"The dog barked loudly"`

    Use the **Viterbi algorithm** to find the most probable sequence of POS tags.

5. **Output:**
    The predicted POS tag sequence.

    Example Output:

```
Sentence: The dog barked loudly
Predicted Tags: DET NOUN VERB ADV
```

Expected Deliverables:

- HMM model implementation (states, transitions, emissions)
- Viterbi algorithm for decoding
- Evaluation on test data (accuracy, confusion matrix)

Why HMM is used here:

- The **POS tags** are *hidden states*.
- The **words** are *observed outputs*.
- The **probability of a tag sequence** depends on previous tags (Markov property).
- The **probability of a word** depends on its tag (emission probability).